MODULE 05

NOTE: Watch This Module Via YT as Just Reading this Theory You Might Get Confused. No Numerical are there in this module, No Need To Worry About Numerical

Q1: What does it mean for an algorithm to run in polynomial time, and why is it considered reasonable?
A: An algorithm runs in polynomial time if its worst-case running time is $O(n^k)$ for some constant k. This includes complexities like $O(1)$, $O(n \log n)$, $O(n^2)$, or $O(n^3)$. It's considered reasonable because it scales efficiently with input size, unlike exponential or factorial time (e.g., $O(2^n)$, $O(n!)$), which quickly become impractical.

Q2: What is the difference between optimization and decision problems, and how are they related?
A: An optimization problem seeks the best (optimal) solution to a problem, like minimizing cost or maximizing profit (e.g., 0-1 Knapsack, MST, Traveling Salesman). A decision problem asks a yes/no question about the problem (e.g., "Is there a Hamiltonian cycle of weight ≤ k?"). Many problems have both forms—solving the decision version often helps tackle the optimization one.

Q3: What are the classes P and NP, and how do they differ in computational theory?

- Definition: P is the class of problems that can be solved by a deterministic Turing machine (i.e., a regular algorithm) in polynomial time, meaning the time it takes to solve the problem is $O(n^k)$ for some constant k, where n is the size of the input.
- Key Idea: If a problem is in P, it means we can find the correct solution efficiently and reliably, using a standard algorithm.
- Example Problems in P:
    - Fractional Knapsack Problem – Greedy algorithm solves it in O(n log n).
    - Minimum Spanning Tree (MST) – Kruskal's or Prim's algorithm.
    - Sorting Algorithms – Merge Sort, Quick Sort, etc.
    - Shortest Path – Dijkstra's algorithm.
- Deterministic Algorithm:
  An algorithm that, given the same input, always follows the same steps and produces the same output.

- Class NP (Nondeterministic Polynomial Time):
    - Definition: NP is the class of decision problems where:
        - If the answer is YES, there exists a certificate (proof) or solution that can be verified in polynomial time by a deterministic algorithm.
        - OR, equivalently, problems solvable in polynomial time by a nondeterministic machine—a theoretical machine that can "guess" the correct path instantly.
    - Key Idea: NP problems may not be efficiently solvable, but if someone hands you a solution, you can verify it quickly (in polynomial time).
    - Example Problems in NP:
        - Traveling Salesman Problem (TSP) – Decision version: "Is there a tour of total weight ≤ k?"
        - Graph Coloring – Can a graph be colored using k colors without adjacent nodes sharing a color?
        - Satisfiability (SAT) – Is there an assignment of truth values that makes a Boolean formula true?
        - 3-CNF SAT – Special case of SAT where each clause has exactly 3 literals.

📌 Relationship Between P and NP:

- All problems in P are also in NP:
  If you can solve a problem efficiently (P), then you can obviously verify that solution efficiently (NP).
  ⇒ So, P ⊆ NP.
- The Big Question: Does P = NP?
  - This is one of the greatest unsolved problems in computer science.
  - If P = NP, it means every problem whose solution can be verified quickly can also be solved quickly.
  - Most computer scientists believe P ≠ NP, but there is no proof yet.
  -

Q4: What does it mean for a problem to be NP-Hard?
A: A problem is NP-Hard if *every problem in NP can be reduced to it in polynomial time*. That means solving it efficiently would let us solve all NP problems efficiently.
➡️ But: NP-Hard problems aren't necessarily in NP — they might not even be decision problems (they could be optimization problems too).
Example: Hamiltonian Cycle, Traveling Salesman Problem (TSP).

Q5: What does it mean for a problem to be NP-Complete?
A: A problem is NP-Complete if it is both:

- In NP (its solution can be verified in polynomial time), and
- NP-Hard (every NP problem reduces to it).

It's like the "hardest of the easy-to-check" problems.
➡️ If you solve one NP-Complete problem efficiently, you've cracked all of NP.
Example:

- SAT (Boolean Satisfiability Problem) – the first proven NP-Complete problem (Cook's Theorem).
- 3-CNF-SAT, Clique, Vertex Cover.

Q6: What is a reduction in computational complexity?
A: A reduction is a method to convert one problem R into another problem Q, such that solving Q also gives a solution to R.

- This is done in polynomial time, called a polynomial-time reduction.
- If R reduces to Q, then R is no harder than Q.

Example:
You can reduce lcm(m, n) to gcd(m, n) using the formula:

```
lcm(m, n) = m * n / gcd(m, n)
```

Q: What is Cook's Theorem?
A:
Cook's Theorem (aka Cook-Levin Theorem) proves that the Boolean Satisfiability Problem (SAT) is NP-Complete.
This means:

- Every problem in NP can be reduced to SAT in polynomial time.
- So, if SAT can be solved in polynomial time, P = NP.

🧠 Impact: It was the first known NP-Complete problem and it laid the foundation for the whole theory of NP-completeness.

Q7: What is the Circuit Satisfiability Problem (CIRCUIT-SAT)?
A: Given a Boolean combinational circuit built from AND, OR, NOT gates, is there an input assignment that makes the output 1?

- It asks: "Is the circuit satisfiable?"
- CIRCUIT-SAT = {⟨C⟩ | C is a satisfiable Boolean circuit}
- Brute-force runs in $\Omega(2^k)$ where k = number of inputs. Not polynomial.

🔧 Hardware Insight: If a subcircuit always outputs 0, you can replace it with a simpler structure → useful in hardware optimization.

Q8: How does CIRCUIT-SAT reduce to SAT?
A: CIRCUIT-SAT can be translated into a SAT formula using logical expressions to describe the gates and wires of the circuit.

- Every gate's function is expressed as a logical constraint.
- The output condition (must be 1) becomes the final clause.
- Whole process runs in polynomial time.

So:
CIRCUIT-SAT $\leq_p$ SAT
This proves SAT is NP-Hard, and since SAT ∈ NP → SAT is NP-Complete.

Q9: How do we reduce 3-CNF SAT to CLIQUE?
A: To reduce 3-CNF-SAT (a SAT problem where each clause has exactly 3 literals) to CLIQUE, we:

1. Create a node in the graph for each literal in each clause.
2. Connect nodes from different clauses (but not complementary literals like x and $\neg x$).
3. Ask: Does this graph contain a k-clique, where k is the number of clauses?

➡️ If yes, there's a satisfying assignment to the formula.
➡️ So, formula satisfiable ⇔ k-clique exists in the graph.
🧩 Example:
A formula with 4 clauses will map to a graph where we search for a 4-clique.

Q10: What is the CLIQUE problem?
A: CLIQUE = {⟨G, k⟩ : G is a graph that contains a k-clique}

- A clique is a set of nodes that are all connected to each other.
- CLIQUE is a classic NP-Complete problem.